

Documentación Orders Service

Erick Sebastian Meneses Navarro - Axity

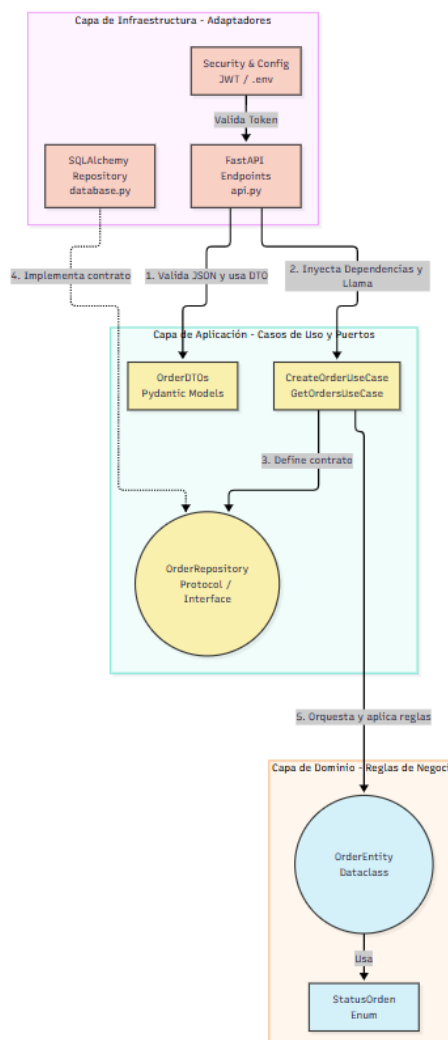
1. Visión General del Proyecto

Este documento describe la arquitectura, diseño y configuración del **Microservicio de Gestión de Órdenes (Order Service)**. El sistema fue diseñado utilizando los principios de la **Arquitectura Hexagonal (Puertos y Adaptadores)**, asegurando un alto nivel de escalabilidad, seguridad y mantenibilidad.

2. Diagramas Arquitectónicos

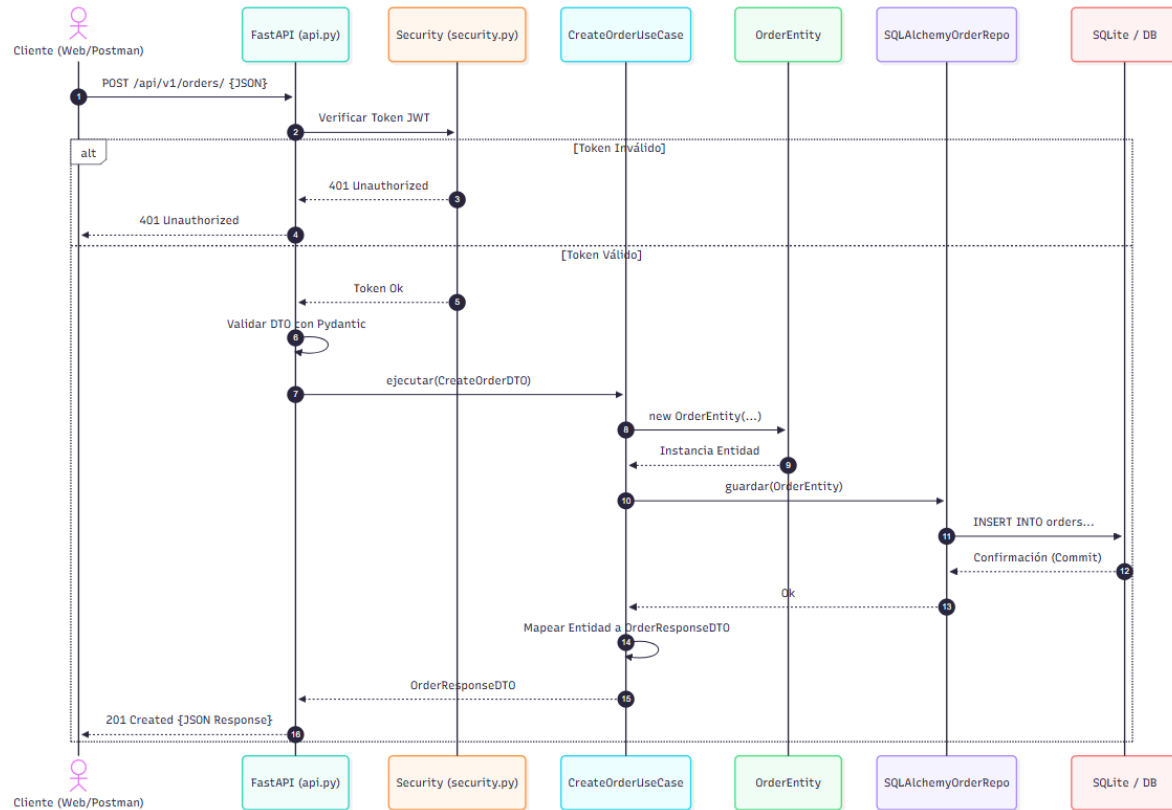
2.1 Diagrama de Arquitectura Hexagonal

El siguiente diagrama detalla cómo las capas exteriores (Infraestructura) se conectan hacia el núcleo (Dominio) mediante los puertos definidos en la capa de Aplicación.



2.2 Flujo de Creación de una Orden (Secuencia)

El siguiente diagrama ilustra el viaje paso a paso de una petición HTTP desde que el cliente la envía hasta que la base de datos la almacena.



3. Arquitectura del Sistema

El proyecto sigue una estricta Regla de Dependencia: el código exterior depende del interior, nunca al revés.

3.1 Capa de Dominio (app/domain/)

Es el núcleo del sistema. * **Entidades**: Contiene la lógica de negocio pura (OrderEntity). * **Agnóstico**: No tiene dependencias de frameworks web (FastAPI) ni ORMs (SQLAlchemy).

3.2 Capa de Aplicación (app/application/)

Coordina las acciones y define los contratos. * **Puertos (Interfaces)**: Define los protocolos (OrderRepository) que el mundo exterior debe cumplir. * **Casos de Uso**: Orquestadores de la lógica (CreateOrderUseCase, GetOrdersUseCase). * **DTOs**: Modelos de transferencia de datos con Pydantic para limpiar las entradas y salidas de la API.

3.3 Capa de Infraestructura (app/infrastructure/)

Donde la aplicación interactúa con el mundo real. * **Adaptadores Primarios:** Controladores REST (FastAPI). * **Adaptadores Secundarios:** Conexión a la base de datos (SQLAlchemy y SQLite). * **Seguridad:** Implementación de JWT (JSON Web Tokens) y gestión de secretos (.env). * **Wiring (Ensamblaje):** Inyección de dependencias que conecta los Adaptadores con los Puertos.

4. Estructura del Repositorio

order-service-hexagonal/	
├── .github/workflows/	# Pipelines de CI/CD (GitHub Actions)
├── app/	# Código Fuente Principal
│ ├── domain/	# Reglas de negocio puras
│ ├── application/	# Orquestación y Contratos (Puertos)
│ └── infrastructure/	# FastAPI, SQLAlchemy, Seguridad
├── tests/	# Suite de Pruebas
│ ├── unit/	# Tests sin BD (Dominio y Mocks)
│ └── integration/	# Tests E2E, API y Contratos de BD
├── alembic/	# Scripts de Migración de Base de Datos
├── .env	# Variables de entorno y secretos
├── .pre-commit-config.yaml	# Reglas de Linter y Formato
├── alembic.ini	# Configuración de Migraciones
├── Dockerfile	# Receta de Contenerización Multistage
├── pyproject.toml	# Gestión de Dependencias (Poetry)
└── README.md	# Documentación rápida

5. Ecosistema Tecnológico

5.1 Backend & Web Framework

- **Lenguaje:** Python 3.12+
- **Framework Web:** FastAPI (Asíncrono, basado en estándares OpenAPI).
- **Servidor ASGI:** Uvicorn.

5.2 Base de Datos & ORM

- **ORM:** SQLAlchemy 2.0 (Syntax Mapped para tipado estricto).
- **Motor:** SQLite (Para entornos locales). Intercambiable por PostgreSQL.
- **Migraciones:** Alembic (Manejo de esquemas con rutas absolutas).

5.3 Seguridad y Observabilidad

- **Autenticación:** Implementación nativa de OAuth2PasswordBearer (JWT).
- **Secretos:** pydantic-settings usando SecretStr para prevenir filtraciones en logs.
- **Métricas:** Middleware inyectado para registrar X-Process-Time en las cabeceras HTTP.

6. Calidad de Código y CI/CD

El proyecto aplica una política de Tolerancia Cero a Errores mediante un pipeline automatizado.

6.1 Herramientas Locales (Pre-commit)

- **Ruff:** Linter y Formatter principal (Reemplaza a Flake8, Black e Isort). Garantiza cumplimiento del PEP 8.
- **MyPy:** Análisis de Tipado Estático para prevenir errores en tiempo de ejecución.

6.2 Estrategia de Pruebas (Pytest)

El proyecto cuenta con un **100% de Cobertura de Código (Coverage)**: * **Pruebas Unitarias:** Validan la matemática del dominio y el enrutamiento de los casos de uso usando Mocks (FakeOrderRepository). * **Pruebas de Contrato:** Garantizan que las bases de datos respeten los métodos exigidos por los Puertos usando IDs dinámicos (uuid). * **Pruebas E2E (End-to-End):** Simulan peticiones web, evaluando desde la autenticación HTTP (TestClient) hasta la persistencia en BD (Happy y Sad Paths).

6.3 Pipeline CI/CD (GitHub Actions)

Se dispara ante cada push a la rama main: 1. Sincronización de dependencias (poetry lock). 2. Auditoría de seguridad contra vulnerabilidades CVE (pip-audit). 3. Validación de Linter y Tipado Estricto. 4. Migración de BD temporal y ejecución de Pytest. 5. Construcción de Imagen Docker final.

7. Despliegue y Dockerización

El servicio se distribuye utilizando un **Dockerfile Multistage**: * **Stage 1 (Builder):** Descarga dependencias y construye el entorno usando Poetry en un modo que actúa solo como gestor de dependencias, (package-mode = false). * **Stage 2 (Production):** Construye la imagen final (python:3.12-slim). * **Hardening (Seguridad):** Se crea el usuario myuser sin privilegios (non-root). La base de datos y la aplicación se ejecutan bajo permisos restringidos. * **Arranque:** Encadenamiento de comandos (alembic upgrade head && uvicorn) para garantizar persistencia antes del tráfico.

8. Comandos Frecuentes

Desarrollo Local:

```
poetry install
poetry run pre-commit install
poetry run alembic upgrade head
poetry run uvicorn app.main:app --reload
```

Pruebas y Calidad:

```
poetry run ruff check . --fix  
poetry run pytest -v --cov=app
```

Docker:

```
docker build -t order-service:v1 .  
docker run -p 8000:8000 order-service:v1
```

Documentación fast API

<http://localhost:8000/docs>